Visitor

Visitor · Pretty-print / evaluate expression AST · Build with Waqar

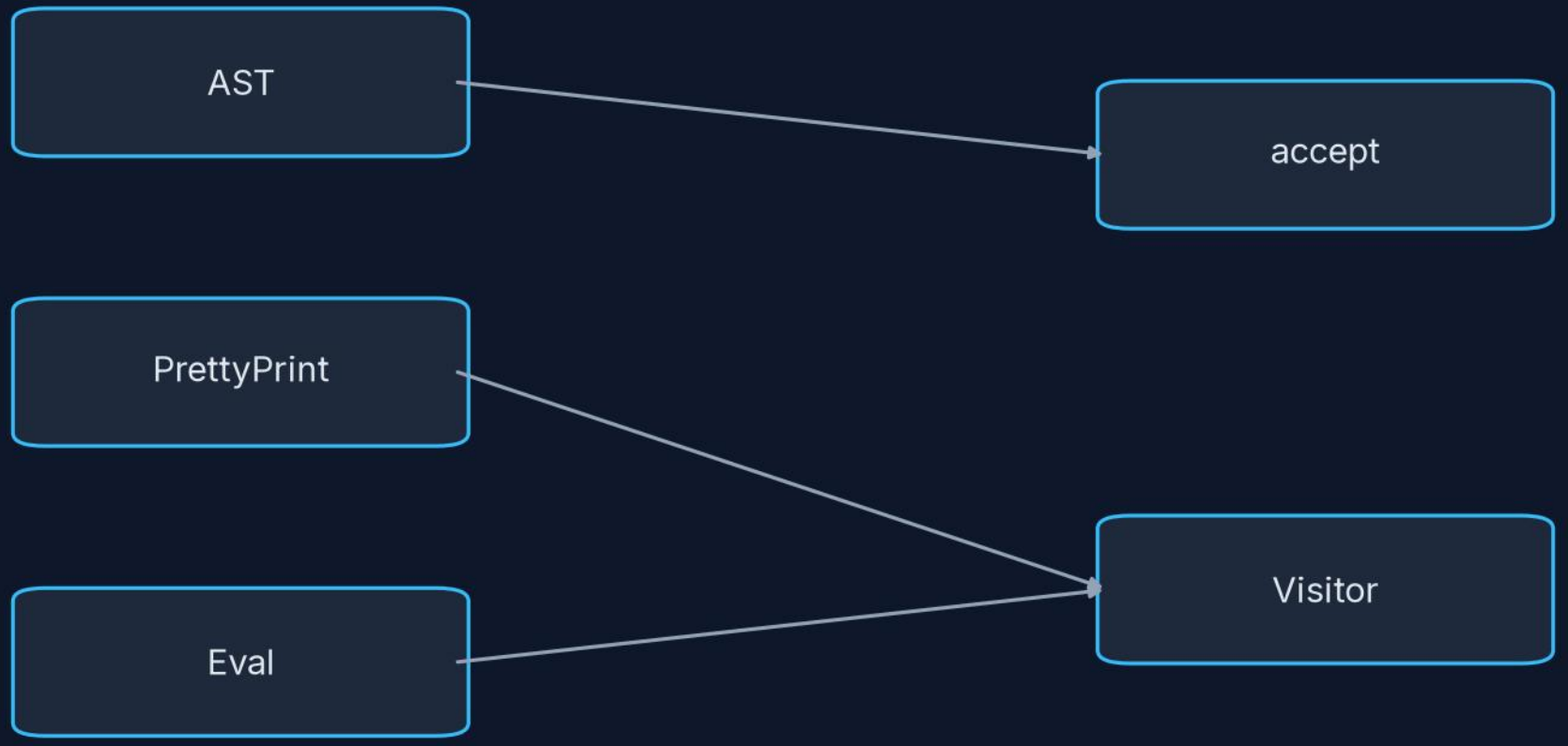
Today's Hook

- Today's force: Pretty-print / evaluate expression AST
- We earn Visitor by feeling the naive pain first.
- Demo main prints a real JDK 25 trace (happy + failure).
- Key types: Expr, Num, Add, Mul

The Problem

- Scenario: Pretty-print / evaluate expression AST
- Constraints: solve «Pretty-print / evaluate expression AST» with Visitor — keep the demo runnable and the failure path observable.
- Naive pain: Naive version fails to cleanly solve: Pretty-print / evaluate expression AST
- Success looks like: (2+3)

Structure



Visitor: Pretty-print / evaluate expression AST

Real-World Scenario

- Running example: Pretty-print / evaluate expression AST
- Key types: Expr, Num, Add, Mul
- Happy-path stdout cue: (2+3)
- Failure-path stdout cue: Failure path is asserted in ProblemTest (injected fault / invalid input)

Naive Code

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;

import java.util.*;

public final class NaiveAst {
  public static String show(String kind, int a, int b){
    if("add".equals(kind)) return "("+a+"+"+b+")";
    return "("+a+"*"+b+")";
  }
}
```

Why Naive Hurts

- Naive version fails to cleanly solve: Pretty-print / evaluate expression AST
- Without Visitor, call sites absorb the wrong details.
- Failure mode to show on camera: Failure path is asserted in ProblemTest (injected fault / invalid input)
- New requirements force shotgun edits across the naive code.

Solution

```
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

sealed interface Expr permits Num, Add, Mul { <R> R accept(Visitor<R> v); }
record Num(int v) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Add(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Mul(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
interface Visitor<R> { R visit(Num n); R visit(Add a); R visit(Mul m); }
final class Pretty implements Visitor<String> {
  public String visit(Num n){ return Integer.toString(n.v()); }
  public String visit(Add a){ return "("+a.l().accept(this)+"+"+a.r().accept(this)+")"; }
  public String visit(Mul m){ return "("+m.l().accept(this)+"*"+m.r().accept(this)+")"; }
}
final class Eval implements Visitor<Integer> {
  public Integer visit(Num n){ return n.v(); }
  public Integer visit(Add a){ return a.l().accept(this)+a.r().accept(this); }
  public Integer visit(Mul m){ return m.l().accept(this)*m.r().accept(this); }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;
import java.util.*; import java.util.concurrent.*;
public final class ProblemDemo {
  public static void main(String[] args) throws Exception {
    System.out.println("=== Visitor: AST pretty-print / eval ===");
    System.out.println("--- naive ---"); System.out.println(NaiveAst.show("add",2,3));
System.out.println("--- visitor ---");
Expr e=new Mul(new Add(new Num(2), new Num(3)), new Num(4));
System.out.println(e.accept(new Pretty()));
System.out.println("value="+e.accept(new Eval()));
  }
}
```

Walkthrough Output

```
=== Visitor: AST pretty-print / eval ===
--- naive ---
(2+3)
--- visitor ---
((2+3)*4)
value=20
```

Step-by-Step Trace

- Match stdout to stages — look for: (2+3)
- Then the injected failure: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Re-run: java -cp … 25-visitor ProblemDemo (see episode folder).
- Pause the video and predict the next printed line.

Modern Java 25 / Zazr

```java
package com.buildwithwaqar.patterns.gof.behavioral.visitor.problem;

import java.util.*;
import java.util.concurrent.*;
import java.util.concurrent.atomic.*;
import java.util.function.*;

sealed interface Expr permits Num, Add, Mul { <R> R accept(Visitor<R> v); }
record Num(int v) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Add(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
record Mul(Expr l, Expr r) implements Expr { public <R> R accept(Visitor<R> v){ return v.visit(this); } }
interface Visitor<R> { R visit(Num n); R visit(Add a); R visit(Mul m); }
final class Pretty implements Visitor<String> {
  public String visit(Num n){ return Integer.toString(n.v()); }
  public String visit(Add a){ return "("+a.l().accept(this)+"+"+a.r().accept(this)+")"; }
  public String visit(Mul m){ return "("+m.l().accept(this)+"*"+m.r().accept(this)+")"; }
}
final class Eval implements Visitor<Integer> {
  public Integer visit(Num n){ return n.v(); }
  public Integer visit(Add a){ return a.l().accept(this)+a.r().accept(this); }
  public Integer visit(Mul m){ return m.l().accept(this)*m.r().accept(this); }
}
```

Trade-offs

- Use Visitor when «Pretty-print / evaluate expression AST» repeats and a simpler feature does not erase it.
- Refuse extra types when a sealed switch or a function value already suffices.
- Teaching demos omit production metrics, retries, and multi-region clocks.
- Interview soundbite: state the force behind Visitor in product language.

Related Patterns

- Name the pattern usually confused with Visitor and separate the forces.
- Sketch the roles for «Pretty-print / evaluate expression AST» from memory in sixty seconds.
- Point to the next episode in this season on the playlist map.
- Link this episode's series/ folder in the YouTube description.

Recap

- Problem: Pretty-print / evaluate expression AST
- Naive pain → Visitor boundary → Demo stdout.
- Remember the failure cue: Failure path is asserted in ProblemTest (injected fault / invalid input)
- Break a ProblemTest assertion, watch it fail, restore it.